What is this project about?

In one sentence

You are replacing an old, flawed data structure (Bloom Filter) used to store network forensic logs with a smarter one (Cuckoo Filter) that can delete old records — and then proving using a real dataset that your version wastes less memory and never loses evidence.

Domain tags

Software Defined Networking (SDN) Network Forensics Information Security Data Structures (DSA)

The two reference papers — what they do

Paper 1 — Varun Sharma (IIIT Hyderabad)

Title: Optimizing Forensic Data Availability and Retention of SDN Forensic Logs by Using Bloom Filter

What it did: Sharma used Bloom Filters to compress and store SDN network logs. He ran Mininet (a network simulator) with 1 controller + 6 switches + 4 hosts, captured pcap files using tcpdump, then inserted packet data (source IP, destination IP, packet size) into Bloom Filters. He showed that 1024 GB of raw logs could be compressed to just ~52 MB using a Bloom Filter.

The gap he left open: At the very end of his paper (Future Work section), he literally says: *"we can plan to experiment on optimizing the bloom filter application by using… cuckoo filter."* — YOUR PROJECT IS EXACTLY THIS FUTURE WORK

Paper 2 — Shesha Shila Bharadwaj (IIIT Hyderabad)

Title: Network Management Framework for Network Forensic Analysis

What it did: This paper introduced the concept of tracing malicious packets back to their source using Bloom Filters stored at routers. It proposed two methods — Distributed Management (each router queries its own Bloom filter) and Centralized Management (all filters sent to a central NMS). The key idea: a victim extracts an "excerpt" (fragment) from a malicious packet and queries each device to find where it passed through.

Why it matters to your project: This is where the concept of **traceability** comes from. Your professor's question "how do you trace back to source?" — the answer is in this paper.

Answering the professor's question: "How do you trace back to source?"

How do you trace back to source without Mininet/a real network?

Answer to give:
"Traceability in our project is defined as *evidence availability* — the ability to successfully retrieve a forensic artifact after a set period of time.

In Shesha Shila's paper, traceability meant following a packet's path through physical routers. In our offline analysis version, it means: if an attack packet was inserted into our filter 7 days ago, can we still find it today?

— With a **Bloom Filter** (Varun Sharma's approach): when the filter fills up, it must be reset entirely.

The attack packet is lost. Traceability = 0% after a wipe.

— With our **Cuckoo Filter**: when capacity is reached, we delete only the oldest 10% of entries using the delete() function. The attack packet from 7 days ago survives. Traceability = preserved.

We simulate this using the CIC-IDS2017 dataset — we feed packets in, hit capacity, and compare query success rates between both filters."

The core problem: why Bloom Filters fail

Issue	Bloom Filter (Existing)	Cuckoo Filter (Your Solution)
Deletion	Cannot delete individual items	Supports delete() per item
When full	Must wipe entire filter — all evidence lost	Delete oldest entries only — evidence preserved
GDPR/Privacy	Cannot selectively remove expired data	Can expire data on a schedule
Memory	Fixed, wastes space for empty slots	More efficient per entry
False negatives	None	None (with Victim Stash buffer)

Your 4 objectives — what you need to build

1

Write Python code for a standard Bloom Filter AND a Cuckoo Filter side-by-side for comparison

2

Download and process the CIC-IDS2017 dataset — feed packet records (src IP, dst IP, size) into both filters to simulate high-volume traffic

3

Program the Cuckoo Filter to auto-delete the oldest 10% of entries when capacity is reached

(the "Sliding Window" — this is the feature Bloom Filters *cannot* do)

4

Measure and graph two things: **Memory Usage (RAM in MB)** and **Data Availability / Traceability (% of queries that succeed)**

Grading breakdown (as told by professor)

Component	Marks
Bloom Filter Python implementation	20%

CIC-IDS2017 dataset processing	20%
Cuckoo Filter with sliding window deletion	20%
Measurement graphs (memory + data availability)	20%
End-of-project report + presentation + viva	20%

Tech stack (no hardware needed)

Tool	Purpose
Python 3.8+	All coding — both filters, analysis scripts
mmh3 library	MurmurHash — needed for Cuckoo Filter hashing
Pandas	Load and parse the CIC-IDS2017 CSV dataset
Matplotlib	Plot memory usage and traceability graphs
CIC-IDS2017 dataset	Free download — University of New Brunswick

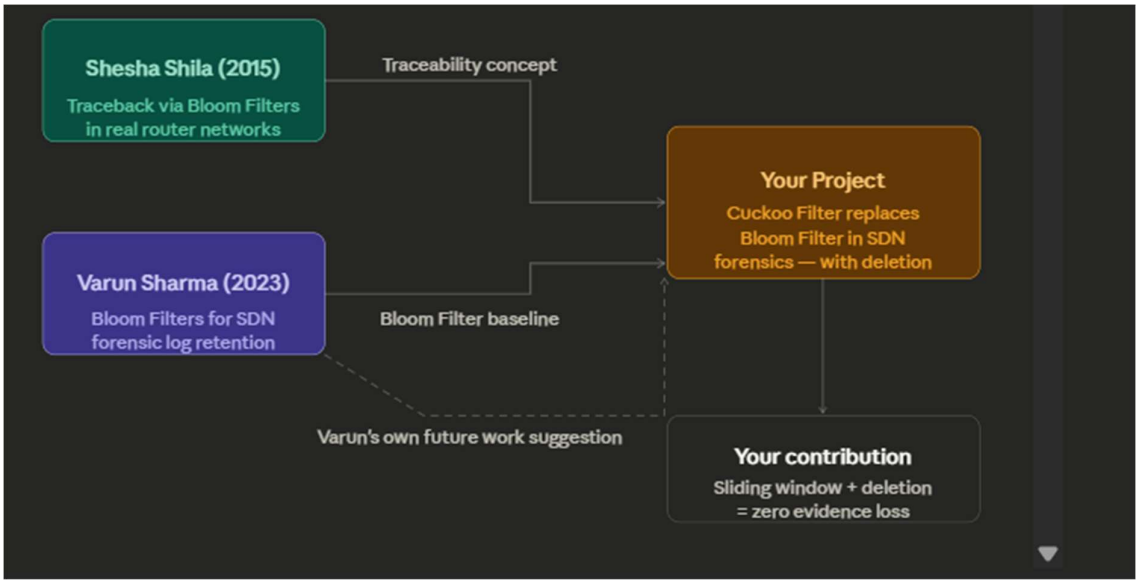
Now let me also show you the big picture of how the three things (your project + both papers) connect to each other:

V

visualize

V

visualize show_widget



Here's a quick summary of the most important things to remember:

Your project in plain language: Varun Sharma proved Bloom Filters save storage for SDN logs. But he admitted at the end of his own paper that Bloom Filters have a fatal flaw — they can't delete data. Your project is literally his suggested next step: replace it with a Cuckoo Filter that *can* delete, prove it's better.

The "traceback to source" question comes from Shesha Shila's paper. In her paper, traceback meant following a packet through physical routers. In your project (since you're doing it offline), traceback means: *"can I still find evidence of an old attack packet after the filter fills up?"* With Bloom Filters — no, it gets wiped. With your Cuckoo Filter — yes, because you only delete the oldest entries.

What you actually build:

1. Python code for both filters
2. Feed the CIC-IDS2017 dataset into both
3. Hit capacity — show Bloom Filter wipes everything, Cuckoo Filter keeps recent evidence
4. Plot memory usage and evidence survival rate graphs